Annota

Audacity-Style Web Audio Editor

Complete Documentation and Usage Guide

Waad Ben Kheder

June 28, 2026

A fully client-side, browser-based audio and video annotation tool built with TypeScript and Vite.
All audio editing happens in the browser — no backend, no server-side processing.
Supports local audio/video files, remote URLs, and YouTube videos.

Contents

1 Introduction **4**
1.1 Quick Start . . . 4
1.2 Build Commands . . . 4

2 User Interface Overview **5**
2.1 Menu Bar . . . 5
2.2 Toolbar Groups . . . 6

3 Loading Media **6**
3.1 Audio Files . . . 6
3.2 Video Files . . . 6
3.3 Load from URL . . . 6
3.4 YouTube Videos . . . 7
3.5 Drag and Drop . . . 7
3.6 Multiple Tracks . . . 8

4 Playback **8**
4.1 Transport Controls . . . 8
4.2 Playback Speed . . . 8
4.3 Seeking . . . 8

5 Visualization **8**
5.1 Waveform . . . 8
5.2 Spectrogram . . . 9
5.3 Zoom and Scroll . . . 9
5.4 Dark / Light Theme . . . 9

6 Selection and Editing **9**
6.1 Creating a Selection . . . 9
6.2 Clipboard Operations . . . 10
6.3 Exporting Audio . . . 10

7 Audio Effects **10**
7.1 Fade In / Fade Out . . . 10
7.2 Normalize . . . 10
7.3 Adjust Gain . . . 10
7.4 Parametric EQ . . . 11
7.5 Compressor / Limiter . . . 11
7.6 Reverb . . . 11
7.7 Noise Reduction . . . 11
7.8 Speed / Pitch Change . . . 11
7.9 Saturation . . . 11
7.10 Resample . . . 12

8 Analysis **12**
8.1 Spectrogram Analysis . . . 12
8.2 Mel Filterbank . . . 12
8.3 MFCC (Mel-Frequency Cepstral Coefficients) . . . 12
8.4 Save Analysis as Image . . . 13

9 Label Track (Annotations) 13
9.1 Label Types . . . 13
9.2 Creating Labels . . . 13
9.3 Editing Labels . . . 13
9.4 Categories . . . 13
9.5 Export Formats . . . 13
9.6 Import . . . 13

10 Per-Track Controls 14
10.1 Volume . . . 14
10.2 Pan . . . 14
10.3 Mute and Solo . . . 14
10.4 Resizable Track Heights . . . 14
10.5 Mixdown Export . . . 14

11 Undo / Redo 14

12 Project Save and Load 15
12.1 Saving . . . 15
12.2 Auto-Save . . . 15
12.3 Loading . . . 15

13 Snap to Grid 15

14 Keyboard Shortcuts 16

15 Technical Architecture 16
15.1 Technology Stack . . . 16
15.2 File Structure . . . 16
15.3 Canvas Architecture . . . 17
15.4 Viewport System . . . 18
15.5 Waveform Mipmap . . . 18
15.6 STFT and Spectrogram . . . 18
15.7 FFT Implementation . . . 19
15.8 Web Worker . . . 19
15.9 Theme System . . . 19
15.10 Undo System . . . 19
15.11 IndexedDB Persistence . . . 20

16 Performance Considerations 20

17 Browser Requirements 21

18 Limitations and Known Issues 21

Introduction

Annota is a web-based audio and video annotation tool inspired by Audacity. It runs entirely in the browser with no server-side processing. It supports loading local audio and video files, fetching media from remote URLs, and embedding YouTube videos for annotation. The application is built with:

- **TypeScript** (strict mode) with ES module imports/exports
- **Vite** as the build tool and development server
- **HTML5 Canvas API** for all graphical rendering (waveform, spectrogram, timeline, segments)
- **Web Audio API** for audio decoding (including audio extraction from video containers), playback, and real-time processing
- **YouTube IFrame Player API** for embedded YouTube video playback and synchronization
- **IndexedDB** for persistent project storage
- **Web Workers** for non-blocking STFT computation
- **Zero runtime dependencies** — FFT, WAV encoding, color maps, effects, and all UI are implemented from scratch

Quick Start

Annota requires **Node.js 22+** and **npm**.

1. Install dependencies:

```
npm install
```

2. Start the development server:

```
npm run dev
```

3. Open `http://localhost:5173` in a modern browser.
4. Click **Load** or drag an audio/video file onto the editor, or use **File > Load from URL** or **File > Load YouTube Video**.

Note: Opening `index.html` directly via `file://` will not work. ES modules and Web Workers require an HTTP server, which the Vite dev server provides.

Build Commands

Supported audio formats depend on the browser's `decodeAudioData` implementation. Most browsers support WAV, MP3, OGG, FLAC, M4A, AAC, and WebM. Video files (MP4, WebM) are also supported — the audio track is extracted for waveform display and the video is shown in a preview panel.

Command	Description
`npm run dev`	Start Vite development server with hot reload
`npm run build`	Production build to `dist/` (minified, tree-shaken)
`npm run typecheck`	Run TypeScript type checking without emitting

Table 1: Available npm scripts

User Interface Overview

The interface is arranged vertically in the following zones:

1. **Menu Bar** (top): File, Edit, Tracks, Effects, Analysis, Help menus.
2. **Toolbar** (46px): Quick-access buttons for file ops, transport, zoom, clipboard, snap, and theme toggle.
3. **Time Ruler** (30px): Adaptive time axis with tick marks and labels.
4. **Tracks Area** (flexible): Vertically stacked, resizable track rows:
   - **Main Audio Track**: Layered canvases showing waveform or spectrogram, selection overlay, and playback cursor. Left panel shows track metadata and controls (volume, pan, mute, solo).
   - **Extra Audio Tracks**: Additional audio files loaded as parallel tracks with independent controls.
   - **Segment Track**: Annotation canvas with segments, speaker badges, and category indicators.
5. **Scrollbar**: Horizontal scrollbar for navigation.
6. **Status Bar** (bottom, 28px): Sample rate, channels, duration, selection range, cursor position, zoom level.

All track rows (audio and segments) are vertically resizable by dragging the handle at the bottom edge of each row.

Menu Bar

Toolbar Groups

Loading Media

Audio Files

Click **Load** or press `Ctrl+O` to open the system file picker. Select any supported audio file (WAV, MP3, OGG, FLAC, M4A, AAC, WebM).

Video Files

Video files (MP4, WebM, OGV, MOV) can be loaded via the file picker or drag-and-drop. When a video file is loaded:

- The audio track is extracted using the Web Audio API's `decodeAudioData`, which supports audio extraction from video containers in most browsers.

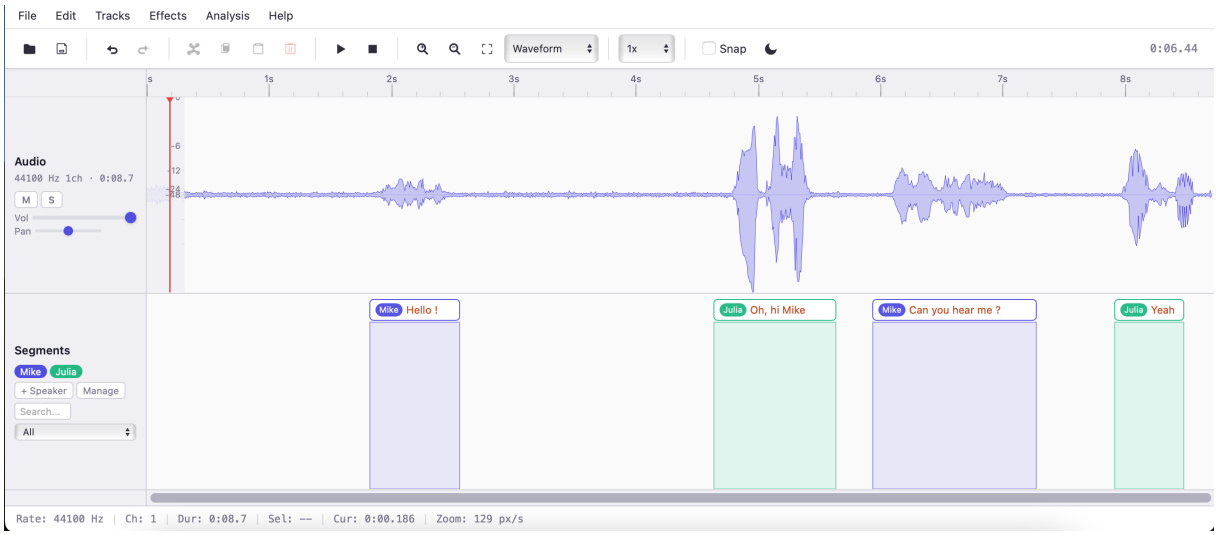

Figure 1: Annota interface overview showing waveform display, segment track with speaker badges, and track controls.

- The waveform and spectrogram are displayed from the extracted audio.
- The video is shown in a **floating preview panel** that is draggable and resizable.
- The video preview can be toggled between three display modes using the **V** key or the toggle button in the panel header:
  1. **Floating**: A draggable panel overlaying the editor (default).
  2. **Inline**: The video replaces the waveform area at the top of the tracks.
  3. **Hidden**: The video is hidden but audio remains active.
- Video playback is synchronized with the audio timeline — play, pause, stop, seek, and playback speed all apply to both audio and video.

Load from URL

Press `Ctrl+U` or use **File** > **Load from URL** to open a dialog where you can enter the URL of an audio or video file. The file is fetched and decoded:

- **Download progress**: For servers that provide a `Content-Length` header, a streaming download progress indicator shows the percentage and MB downloaded (e.g., "Downloading... 45% (12.3 / 27.5 MB)").
- **Video detection**: If the URL points to a video file (detected via content type or file extension), the video preview panel is shown automatically.
- **CORS**: Remote servers must allow cross-origin requests. If blocked by CORS, a clear error message is displayed.

YouTube Videos

Use **File** > **Load YouTube Video** to open a dialog where you can enter a YouTube URL or video ID. The following formats are supported:

- `https://www.youtube.com/watch?v=`*`ID`*

Menu	Items
File	New Project, Open Project, Save Project, Import Audio, Load from URL, Load YouTube Video, Import Segments, Export Audio, Export Segments, Export Mixdown, Export Waveform Image
Edit	Undo, Redo, Cut, Copy, Paste, Duplicate, Delete, Select All, Deselect, Reset Audio
Tracks	Add Audio Track, Append Audio, Separate Channels, Split Stereo to Mono, Combine to Stereo, Mix and Render, Combine Tracks, Manage Speakers, Add Segment, Resample Track
Effects	Fade In, Fade Out, Normalize, Adjust Gain, Noise Reduction, Reverb, Saturation, Parametric EQ, Compressor, Speed/Pitch
Analysis	Spectrum, Spectrogram, Filterbank (Mel), MFCC, Save Analysis Image
Help	Keyboard Shortcuts, About

Table 2: Menu bar structure

Group	Controls
File	Load, Save Project
Edit	Undo, Redo
Clipboard	Cut, Copy, Paste, Delete
Transport	Play, Pause, Stop
Zoom	Zoom In, Zoom Out, Fit, View Mode (Waveform / Spectrogram)
Speed	Playback speed selector (0.5x – 2x)
Snap & Theme	Snap toggle, Dark/Light mode toggle
Time	Current playback time display

Table 3: Toolbar control groups

- `https://www.youtube.com/watch?v=`*`ID`*`&list=`*`LIST`* (only the video ID is extracted)
- `https://youtu.be/`*`ID`*
- `https://www.youtube.com/shorts/`*`ID`*
- A bare 11-character video ID

When a YouTube video is loaded:

- The YouTube IFrame Player API is loaded dynamically from Google's CDN.
- The video is embedded in the floating preview panel with YouTube's native controls.
- A timeline is created matching the video's duration, allowing the full use of the segment track for annotation (e.g., transcription, speaker tagging, event marking).
- The cursor position and segment timecodes are synchronized with the YouTube player's current time. Clicking on the timeline seeks the YouTube player.

- Play, pause, stop, and space bar control the YouTube player. The user can also interact with YouTube's built-in controls, and the annotation UI stays in sync.
- Playback speed changes apply to the YouTube player.
- **Note**: No waveform is available in YouTube mode since raw audio data cannot be extracted from an embedded YouTube video. The timeline shows a flat waveform as a visual placeholder.

Drag and Drop

Drag files directly onto the editor. The drop zone accepts:

- Audio files (WAV, MP3, OGG, FLAC, M4A, AAC, WebM)
- Video files (MP4, WebM, OGV, MOV)
- Segment files (`.txt`, `.json`, `.srt`, `.vtt`, `.xml`, `.stm`, `.tsv`)

Multiple Tracks

- **Add Audio Track** (Tracks menu): Loads a second audio file as a parallel track displayed below the main track. Each track has independent volume, pan, mute, and solo controls. All tracks are vertically resizable.
- **Append Audio** (Tracks menu): Loads a file and concatenates it to the end of the current audio.
- **Separate Channels** (Tracks menu): For stereo files, separates into mono tracks for independent editing within the same project.
- **Split Stereo to Mono** (Tracks menu): Splits a stereo track into two independent mono tracks (left and right channels).
- **Combine to Stereo** (Tracks menu): Opens a dialog to combine two mono tracks into a single stereo track, with selectable left/right channel assignment.
- **Mix and Render** (Tracks menu): Mixes all unmuted tracks down to a single mono track, replacing the current audio.
- **Combine Tracks** (Tracks menu): Opens a dialog to mix multiple tracks together with adjustable balance. Options include looping the shorter track to fill the longer track's duration, volume preview, and output mode (mixdown to single track or keep as separate tracks).
- **Manage Speakers** (Tracks menu): Opens the speaker management dialog for adding, removing, renaming, recoloring, and merging speakers.

Playback

Transport Controls

- **Play** (Space): Starts playback from the cursor position. If a selection exists, plays only the selected range.
- **Pause** (Space during playback): Pauses at the current position.

- **Stop**: Stops playback and resets position to the cursor.

During playback, a red cursor line sweeps across the waveform at 60 fps. The view auto-scrolls to keep the cursor visible.

When a video file is loaded, the `<video>` element is synchronized to the audio playback position. Drift greater than 150 ms is corrected by seeking the video element. In YouTube mode, the YouTube player is the source of truth for time — the annotation cursor tracks the YouTube player's current position, and all transport controls (play, pause, stop, seek) are routed to the YouTube player API.

Playback Speed

The speed selector in the toolbar allows playback at 0.5x, 0.75x, 1x, 1.25x, 1.5x, or 2x speed. This is applied to:

- The Web Audio API's `playbackRate` property (audio files)
- The `<video>` element's `playbackRate` property (video files)
- The YouTube player's `setPlaybackRate` method (YouTube mode)

Seeking

- Click on the **time ruler** to jump to that time position.
- Click on the **waveform area** to set the cursor.
- Press **Home** to jump to the beginning, **End** to jump to the end.

Visualization

Waveform

The waveform view shows amplitude over time. For stereo files, both channels are stacked vertically. Rendering uses a hierarchical peak mipmap for instant response at any zoom level.

A **dB scale** overlay appears on the left edge showing amplitude markings at 0, −6, −12, −24, and −48 dB.

Spectrogram

The spectrogram view shows frequency content over time, computed via Short-Time Fourier Transform (STFT):

- FFT size: 2048 samples
- Hop size: 512 samples
- Window: Hann
- Color map: Magma
- Dynamic range: −90 dB to 0 dB

A **frequency scale** overlay appears on the left edge showing frequency markings in Hz/kHz, ranging up to the actual Nyquist frequency (half the sample rate). A Nyquist label is displayed at the top of the scale.

Spectrogram computation is delegated to a Web Worker with progress reporting. On browsers without Worker support, it falls back to chunked main-thread computation that yields periodically to avoid freezing the UI.

Zoom and Scroll

- **Ctrl+Scroll**: Zoom in/out centered on the mouse position. Supports smooth trackpad zoom with delta accumulation and damping for fine-grained control.
- **Scroll** (no modifier): Horizontal scroll.
- +/− buttons: Step zoom by factor of 2.
- **Fit**: Zoom to show the entire audio file.
- **Arrow keys**: Scroll left/right.

The zoom level is measured in *samples per pixel* (SPP). Minimum SPP is 4 (most zoomed in), maximum is 65536 (most zoomed out). The status bar shows the equivalent pixels-per-second.

Dark / Light Theme

Click the moon icon in the toolbar to toggle between light and dark themes. The theme preference is persisted in `localStorage`. All canvas renderers (waveform, spectrogram, timeline, selection, segments, axis overlays) use theme-aware colors that update immediately on toggle.

Selection and Editing

Creating a Selection

Click and drag on the waveform area to create a time selection. The selected region is highlighted with a semi-transparent overlay with vertical edge markers.

If **Snap** is enabled (checkbox in the toolbar), selection edges are quantized to grid intervals that adapt to the zoom level.

Clipboard Operations

- **Cut** (`Ctrl+X`): Removes the selected audio and places it in the clipboard.
- **Copy** (`Ctrl+C`): Copies the selected audio to the clipboard.
- **Paste** (`Ctrl+V`): Inserts clipboard audio at the cursor position.
- **Duplicate** (`Ctrl+D`): Copies and immediately pastes the selection after the selection end.
- **Delete** (`Delete`): Removes the selected audio without copying.
- **Select All** (`Ctrl+A`): Selects the entire audio duration.

All destructive operations are undoable.

Exporting Audio

Via the File menu or toolbar, a dialog offers:

- **WAV** (lossless): Always available.
- **MP3**: Browser-dependent (requires `MediaRecorder` with audio/mpeg support).
- **Sample rate**: Original or resample to 8000, 16000, 22050, 44100, or 48000 Hz.
- **Mono**: Optionally mix down to mono on export.

Audio Effects

All effects are accessible via the **Effects** menu. Effects operate directly on the audio buffer samples and are undoable. When a selection exists, effects apply only to the selected region; otherwise they apply to the entire file.

Fade In / Fade Out

Applies an amplitude envelope across the selected region. A dialog allows choosing the curve type:

- **Linear**: Straight-line ramp.
- **Exponential**: Quadratic curve (t^2 for fade in, $(1-t)^2$ for fade out).
- **S-Curve**: Smooth sigmoid ($3t^2 - 2t^3$).

Normalize

Peak-normalizes audio to -1 dBFS. Finds the peak amplitude in the target region and applies a uniform gain to bring it to the target level.

Adjust Gain

Opens a dialog to enter a gain value in dB. The gain is applied as a linear multiplier ($10^{dB/20}$). Output is clamped to $[-1, 1]$.

Parametric EQ

A multi-band parametric equalizer with visual frequency response curve:

- Multiple bands, each with frequency (Hz), gain (dB), and Q factor.
- Real-time visual preview of the combined frequency response on a canvas.
- Applies cascaded biquad filters to the audio data.

Compressor / Limiter

Dynamics processing with the following parameters:

Parameter	Range	Default	Unit
Threshold	-60 to 0	-20	dB
Ratio	1:1 to 20:1	4:1	—
Attack	0.1 to 100	10	ms
Release	10 to 1000	100	ms
Knee	0 to 20	6	dB
Makeup Gain	0 to 30	0	dB

Table 4: Compressor parameters

Reverb

Algorithmic reverb with three parameters:

- **Room Size** (0–1): Controls the decay time.
- **Damping** (0–1): High-frequency absorption.
- **Wet/Dry Mix** (0–1): Balance between processed and original signal.

Noise Reduction

Two-step spectral noise gate:

1. **Get Noise Profile**: Select a noise-only region and click to capture the noise spectrum.
2. **Apply**: Select the region to clean. The algorithm subtracts the noise profile from each STFT frame, using a sensitivity parameter (0.5–5.0) to control aggressiveness.

The noise reduction dialog is non-modal, allowing the user to change the selection between steps.

Speed / Pitch Change

Resampling-based speed adjustment (0.25x to 4x). Changing speed also changes pitch proportionally. This modifies the buffer length and sample data.

Saturation

Soft-clipping distortion with a drive parameter (1–10). Applies a hyperbolic tangent waveshaper: $output = \tanh(drive \times input)$.

Resample

Changes the audio sample rate via high-quality interpolation. Available target rates: 8000, 16000, 22050, 44100, 48000 Hz.

Analysis

Analysis features are accessible via the **Analysis** menu. Each analysis opens a floating, draggable panel with a canvas visualization. Analysis requires a selection to define the region to analyze.

Spectrum Analysis

Computes and displays a power spectrum of the selected region using Welch's method:

- **FFT size**: 4096 samples (configurable).
- **Window**: Hann window with 50% overlap.
- **Averaging**: Multiple overlapping segments are averaged to reduce variance.
- **Display**: Line plot of magnitude in dB vs. frequency, with optional log or linear frequency axis.
- **Grid**: dB grid lines and frequency axis labels (Hz/kHz).
- **Fill**: Semi-transparent fill under the spectrum curve.

Spectrogram Analysis

Computes and displays a detailed spectrogram of the selected region in a floating panel. Uses the same STFT parameters as the main spectrogram view (2048 FFT, 512 hop, Hann window) with the Magma colormap.

The implementation uses the function-based FFT:

```
const win = createWindow(fftSize, 'hann');
const real = new Float32Array(fftSize);
const imag = new Float32Array(fftSize);
for (let f = 0; f < numFrames; f++) {
  // Window and copy frame
  for (let i = 0; i < fftSize; i++) {
    real[i] = mono[offset + i] * win[i];
    imag[i] = 0;
  }
  fft(real, imag);
  // Compute magnitudes in dB ...
}
```

Mel Filterbank

Computes mel-scale filterbank energies of the selected region:

- Configurable number of mel filters (default: 40)
- FFT size and hop size parameters
- Displays a 2D heatmap of filter energies over time

MFCC (Mel-Frequency Cepstral Coefficients)

Computes MFCCs from the mel filterbank output:

- Applies log compression to mel energies
- Applies Discrete Cosine Transform (DCT) to extract cepstral coefficients
- Configurable number of coefficients (default: 13)
- Displays a 2D heatmap of MFCC values over time

Save Analysis as Image

Any open analysis panel can be saved as a PNG image:

- Click the save button (floppy disk icon) in the analysis panel header.
- Alternatively, use **Analysis** > **Save Analysis Image** from the menu.
- The canvas content is exported via `canvas.toDataURL('image/png')` and downloaded with an auto-generated filename based on the analysis type.

Segment Track (Annotations)

Segment Types

- **Region segments**: A start–end time range, displayed as a filled rectangle with edge handles, a centered text badge, and optional speaker badge.
- **Point segments**: A single timestamp, displayed as a vertical dashed line with a circle marker and a text badge.

Creating Segments

- **Click** on empty space in the segment track: Creates a point segment at that time.
- **Drag** on empty space in the segment track: Creates a region segment spanning the drag range.
- **Ctrl+B**: Creates a segment at the cursor position, or a region segment matching the current selection.

Editing Segments

- **Click** a segment to select it.
- **Double-click** a segment to edit its text inline.
- **Drag** a segment body to move it (clamped to audio duration).
- **Drag** a region edge handle to resize it (clamped to audio duration).
- **Delete/Backspace** with a segment selected to remove it.
- **Ctrl+T**: Split the selected segment at the cursor position into two segments.
- **Merge**: Select adjacent segments and merge them via the context menu.

Speaker System

Segments can be assigned to speakers. The speaker system provides:

- **Speaker management dialog** (Tracks > Manage Speakers or the Manage button in the segment track header): Add, remove, rename, recolor, and merge speakers.
- **Color-coded badges**: Each speaker is assigned a color from a palette of 8 predefined colors (round-robin assignment). Speaker badges appear on segments showing the speaker's name and color.
- **Speaker assignment**: Right-click a segment to assign it to a speaker via the context menu submenu.
- **Merge speakers**: Select two speakers in the management dialog to merge them — all segments from one speaker are reassigned to the other.
- **Enable/disable**: The speaker system can be toggled on/off via a checkbox in the speaker dialog.

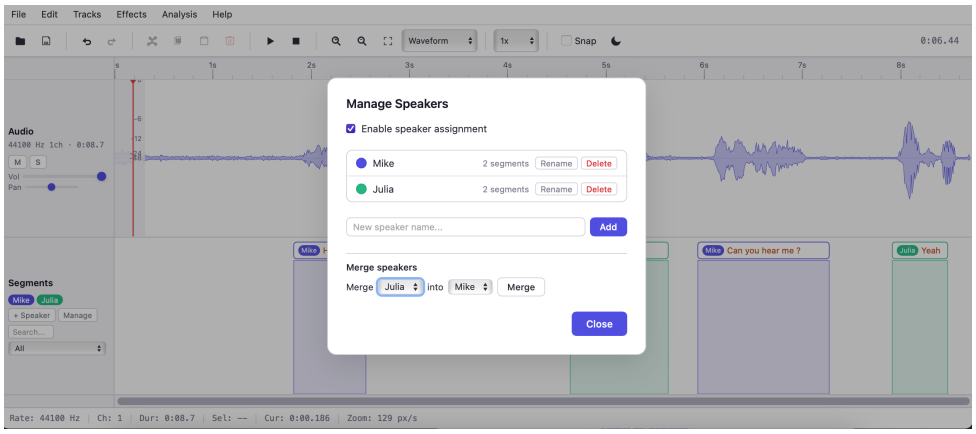

Figure 2: Speaker management dialog showing speaker list with color indicators, rename/delete controls, and merge functionality.

Categories

Segments can be assigned to categories: *speech*, *noise*, *music*, *silence*, or *other*. Each category has a distinct color indicator. The segment track header provides a category filter dropdown and a search box to filter segments by text.

Export Formats

The export dialog presents a grid of 7 format buttons:

Format	Extension	Description
Audacity	`.txt`	Tab-separated: `start\tend\ttext`. Compatible with Audacity's label import.
JSON	`.json`	Array of objects with `start`, `end`, `text`, `speakerId`, `category` fields.
SRT	`.srt`	SubRip subtitle format. Point segments get a default 2-second duration.
WebVTT	`.vtt`	Web Video Text Tracks format for HTML5 `<track>` elements.
STM	`.stm`	NIST Segment Time Mark format: `file channel speaker start end text`.
TSV	`.tsv`	Tab-separated values with configurable columns (start, end, speaker, text, category, duration). A column picker allows selecting which columns to include.
ELAN XML	`.xml`	Simplified ELAN annotation format with time slots and tier structure.

Table 5: Supported segment export formats

Import

Click **File** > **Import Segments** and select a segment file. The format is auto-detected:

- Files starting with `WEBVTT` are parsed as WebVTT.

- Files with `->` timestamp patterns are parsed as SRT.
- Files starting with `[` or `{` are parsed as JSON.
- All other files are parsed as Audacity tab-separated format.

Projects saved with the old label system are automatically migrated to the segment format on load.

Per-Track Controls

Volume

Each track has an independent volume slider (0–1 linear). The main track's volume slider is in the left panel; extra tracks have their own slider.

Pan

Each track has a pan slider (-1 = full left, 0 = center, $+1$ = full right). Panning uses the Web Audio API's `StereoPannerNode`.

Mute and Solo

- **Mute (M)**: Silences the track. The button turns red when active.
- **Solo (S)**: When any track is soloed, only soloed tracks produce output. The button turns accent-colored when active.

The `M` key toggles mute on the main track.

Resizable Track Heights

All track rows (audio and segments) can be vertically resized by dragging the resize handle at the bottom edge of each row. The minimum height is 80px for audio tracks and 50px for the segment track.

Mixdown Export

The Mixdown dialog mixes all unmuted (or, if any track is soloed, all soloed) tracks into a single WAV file. Samples are summed and clamped to $[-1, 1]$.

Undo / Redo

Destructive operations capture a full snapshot (audio buffer + segments + speakers + cursor position) before execution. Up to 30 undo levels are maintained.

- **Ctrl+Z**: Undo the last destructive action.
- **Ctrl+Shift+Z** or **Ctrl+Y**: Redo.

Operations that push to the undo stack include: cut, paste, duplicate, delete, concatenate audio, all effects (fade, normalize, gain, EQ, compressor, reverb, noise reduction, speed change, saturation, resample), channel operations, and segment modifications (add, delete, split, merge, category/speaker assignment).

After undo/redo, the spectrogram is automatically recomputed to stay in sync with the restored audio buffer.

Note: Undo snapshots clone the entire `AudioBuffer` as `Float32Array` channel data. For long recordings, this can consume significant memory.

Project Save and Load

Saving

Press `Ctrl+S` or use the File menu. Enter a project name. The full project state is serialized to IndexedDB:

- Audio buffer data (raw Float32 samples per channel)
- All segments (with categories and speaker assignments)
- All speakers (names, colors)
- Speaker system enabled state
- Viewport state (zoom level, scroll position)
- Cursor position and selection
- View mode
- Extra track data

Auto-Save

After loading audio, an auto-save runs every 30 seconds in the background. The auto-save is stored under a reserved key and can be loaded from the project list.

Loading

Use **File** > **Open Project**. A prompt lists all saved projects (including auto-save if available). The project state is fully restored, including audio, segments, speakers, and viewport. Projects saved with the old label system are automatically migrated to the segment format.

Snap to Grid

Enable the **Snap** checkbox in the toolbar. When enabled:

- Selection edges snap to the nearest grid line.
- Cursor placement snaps when clicking on the timeline.
- The grid interval adapts to the zoom level:

Visible Duration	Snap Interval
$< 1\,\text{s}$	10 ms
$< 5\,\text{s}$	50 ms
$< 15\,\text{s}$	100 ms
$< 60\,\text{s}$	500 ms
$< 300\,\text{s}$	1 s
$\geq 300\,\text{s}$	5 s

Table 6: Snap intervals by zoom level

Keyboard Shortcuts

Technical Architecture

Technology Stack

- **Language**: TypeScript (strict mode, ES2020 target)
- **Build**: Vite 8 with native ES module dev server
- **Module system**: ESNext modules with bundler resolution
- **Type checking**: `tsc -noEmit` (no separate compilation step; Vite handles transpilation)
- **Runtime dependencies**: None (zero `dependencies` in `package.json`)

File Structure

```
Annota/
  index.html              HTML layout, toolbar, dialogs, canvas elements
  styles.css              Light/dark themes via CSS custom properties
  package.json            npm scripts: dev, build, typecheck
  tsconfig.json           TypeScript strict mode configuration
  vite.config.ts          Vite build configuration
  src/
    types.ts              Shared interfaces and type definitions
    utils.ts              Time formatting, color maps, HiDPI canvas
    fft.ts                Radix-2 Cooley-Tukey FFT + window functions
    viewport.ts           Central zoom/scroll state + transforms + smooth trackpad
    zoom
    undo.ts               Snapshot-based undo/redo manager
    audio-engine.ts       Web Audio decoding, playback, editing, ArrayBuffer loading
    waveform.ts           Mipmap peak cache + theme-aware rendering
    spectrogram.ts        STFT computation + rendering
    timeline.ts           Adaptive time ruler with theme colors
    selection.ts          Selection overlay + cursor rendering
    segment-track.ts      Segment model, rendering, speaker badges, import/export (7
    formats)
    speaker-manager.ts    Speaker CRUD, merge, color palette management
    clipboard.ts          Cut/copy/paste for audio and segments
    resampler.ts          Sample rate conversion
    effects.ts            Core effects (fade, normalize, gain)
    worker.ts             Self-contained Web Worker for STFT
    effects/
      eq.ts               Parametric equalizer (biquad cascade)
```

Shortcut	Action
Space	Play / Pause toggle
Ctrl+O	Open file (audio or video)
Ctrl+U	Load from URL
Ctrl+S	Save project
Ctrl+Z	Undo
Ctrl+Shift+Z / Ctrl+Y	Redo
Ctrl+X	Cut selection
Ctrl+C	Copy selection
Ctrl+V	Paste at cursor
Ctrl+D	Duplicate selection
Ctrl+A	Select all
Ctrl+B	Add segment at cursor/selection
Ctrl+T	Split segment at cursor
Ctrl+=	Zoom in
Ctrl+−	Zoom out
Ctrl+F	Fit to view
V	Toggle video preview (floating / inline / hidden)
M	Toggle mute on main track
Delete	Delete selected audio or segment
Backspace	Delete selected segment
Home	Jump to beginning
End	Jump to end
Arrow Left	Scroll left
Arrow Right	Scroll right
Escape	Deselect / close dialog

Table 7: Complete keyboard shortcut reference

```
    compressor.ts       Dynamics compressor with soft knee
    reverb.ts           Algorithmic reverb
    noise-reduction.ts  Spectral noise gate
    time-stretch.ts     Resampling-based speed change
  analysis/
    filterbank.ts       Mel filterbank computation
    mfcc.ts             MFCC extraction (DCT on mel energies)
    spectrum.ts         Welch's method power spectrum analysis + rendering
  ui/
    icons.ts            SVG icon strings
    context-menu.ts     Right-click context menu (with speaker assignment submenu)
    menu-bar.ts         Application menu bar
    theme.ts            Light/dark theme manager
    analysis-panel.ts   Floating analysis panel + save as PNG
  app/
    dom-refs.ts         Typed DOM element references (~90 elements)
    app.ts              Entry point: wires everything together
```

Canvas Architecture

The main track area uses stacked `<canvas>` elements:

1. **Spectrogram canvas**: Pre-rendered STFT image.

2. **Waveform canvas**: Peak-based waveform.
3. **dB / Frequency scale canvas**: Axis overlay (switches based on view mode).
4. **Selection canvas**: Selection highlight region.
5. **Cursor canvas**: Playback cursor line (redrawn at 60 fps).

Only one of waveform or spectrogram is visible at a time. This layering ensures that the frequently-updated cursor does not trigger expensive waveform or spectrogram redraws.

Viewport System

The `Viewport` class is the central coordinator for zoom and scroll state:

- `samplesPerPixel` (SPP): The fundamental zoom unit. Range: 4–65536.
- `scrollSamples`: Horizontal offset in samples.
- `sampleRate`: Audio sample rate for time conversion.
- `zoomByDelta`: Smooth trackpad zoom with delta accumulation and damping, providing fine-grained control on trackpads without overshooting on discrete scroll wheels.

All renderers read from the viewport to determine what portion of the audio is visible. Coordinate transforms:

- `timeToPixel(seconds)` → canvas X position
- `pixelToTime(px)` → time in seconds
- `sampleToPixel(sampleIndex)` → canvas X position

Waveform Mipmap

The waveform renderer pre-computes a peak pyramid (min/max per block) at power-of-2 block sizes from 2 to 65536 samples. Each coarser level is derived from the finer level by comparing pairs:

```
# Pseudocode for mipmap construction
for spp in [2, 4, 8, ..., 65536]:
    if spp == 2:
        compute peaks directly from samples
    else:
        derive from level[spp/2] by comparing pairs
```

This ensures $O(N)$ total construction time and $O(1)$ peak lookup at any zoom level.

STFT and Spectrogram

The spectrogram is computed via Short-Time Fourier Transform:

1. Audio is converted to mono (channel averaging).
2. **A copy** of the mono data is transferred to the Web Worker (the original buffer is preserved).
3. A sliding Hann window (2048 samples, 512-sample hop) extracts frames.
4. Each frame is FFT-transformed using the Radix-2 Cooley-Tukey algorithm.

5. Magnitudes are converted to dB and mapped to colors via the Magma colormap.
6. The result is returned as RGBA pixel data and written to an offscreen canvas.

The visible portion is drawn via `drawImage` with source rectangle clipping, which is GPU-accelerated in most browsers.

FFT Implementation

The FFT module (`fft.ts`) exports a **function-based** API, not a class:

```
// Function signature: operates in-place on real/imag arrays
fft(real: Float32Array, imag: Float32Array): void

// Window function creation
createWindow(size: number, type: WindowType): Float32Array

// Convenience: compute magnitude spectrum
magnitudeSpectrum(real, imag): Float32Array

// Full STFT computation
computeSTFT(samples, fftSize, hopSize, windowType): number[][]
```

Supported window types: `hann`, `hamming`, `blackman`, `bartlett`, `rectangular`.

Web Worker

The `worker.ts` file contains a self-contained copy of the FFT, window functions, and color map. This intentional duplication avoids import issues in the worker context. It handles three message types:

- `computeSTFT`: Full spectrogram computation with progress reporting.
- `computePeakMipmap`: Peak pyramid construction.
- `encodeWAV`: WAV file encoding.

Audio data is copied before being transferred to the worker to prevent detaching the main AudioBuffer's backing memory.

Theme System

The `ThemeManager` class manages light/dark mode:

- Persists preference in `localStorage`.
- Sets `data-theme` attribute on `<html>` for CSS variable switching.
- Provides a `ThemeColors` object with named color values for all canvas renderers.
- On toggle, propagates colors to: WaveformRenderer, SpectrogramRenderer, TimeRuler, SelectionManager, SegmentTrack, and all extra track waveform renderers.

CSS custom properties (defined in `:root` and `[data-theme="dark"]`) handle all DOM element styling.

Undo System

The `UndoManager` maintains two stacks (undo and redo). Before each destructive operation, a snapshot is pushed:

- Audio buffer: All channel data is cloned as `Float32Array` objects (stored as plain objects with `sampleRate`, `numberOfChannels`, `length`, and `channels` fields).
- Segments: Deep-cloned array of all segments with speaker assignments and categories.
- Speakers: Deep-cloned array of all speakers with names and colors.
- Cursor position.

On undo, the current state is pushed to the redo stack and the snapshot is restored by reconstructing an `AudioBuffer` via `AudioContext.createBuffer()`.

IndexedDB Persistence

Projects are stored in IndexedDB under database `annota_projects`. Each project entry contains:

- Timestamp
- Audio data (sample rate, channel count, length, raw Float32 channel arrays)
- Segments array (with categories and speaker assignments)
- Speakers array (names, colors)
- Speaker system enabled state
- Viewport state
- Cursor position
- View mode
- Extra track data

Auto-save writes to a reserved key (`__autosave__`) every 30 seconds.

Performance Considerations

- **Waveform rendering**: The mipmap ensures $O(W)$ rendering cost where W is the canvas width in pixels, regardless of audio length.
- **Spectrogram**: Computed once, then rendered via `drawImage` (near-zero cost per frame).
- **Selection/cursor**: Drawn on dedicated canvases, avoiding full redraw.
- **URL download**: Large files fetched from URLs use streaming download with progress reporting (percentage and MB counters), preventing the UI from appearing frozen during long downloads.
- **Video sync**: Video playback drift is corrected only when it exceeds 150 ms (local video) or 300 ms (YouTube), avoiding constant seeking.
- **Undo memory**: Each snapshot stores the full audio buffer. For a 10-minute stereo 44.1 kHz file, each snapshot is approximately 106 MB. With 30 levels, this can reach 3 GB.

- **Worker offloading**: STFT computation runs in a Web Worker with progress reporting, keeping the UI responsive.
- **Production build**: Vite produces a tree-shaken, minified bundle of approximately 120 KB JS + 15 KB CSS (32 KB + 3 KB gzipped).

Browser Requirements

- **Web Audio API**: Required for audio decoding and playback.
- **Canvas 2D**: Required for all rendering.
- **IndexedDB**: Required for project save/load.
- **Web Workers**: Used for STFT offloading. Falls back to main-thread computation.
- **Fetch API with ReadableStream**: Used for streaming URL downloads with progress. Falls back to buffered download if `response.body` is unavailable.
- **ES2020+**: Modules, `class`, `async/await`, optional chaining, nullish coalescing.
- **Internet access** (optional): Required only for YouTube integration (loads the YouTube IFrame Player API from Google's CDN).

Tested on Chrome 90+, Firefox 88+, Safari 15+, and Edge 90+.

Limitations and Known Issues

- No recording from microphone (could be added via `MediaRecorder`).
- MP3 export depends on browser `MediaRecorder` support and may fall back to WAV.
- The undo system stores full buffer snapshots, which is memory-intensive for large files. A diff-based or command-pattern undo could reduce memory usage.
- Segment track supports only a single tier. Multiple tiers would be useful for complex annotation workflows.
- No spectral editing (painting on the spectrogram to modify audio).
- No real-time effect preview — all effects are applied offline to the buffer.
- **YouTube mode**: No waveform is available since raw audio cannot be extracted from an embedded YouTube video. The timeline shows a flat waveform as a visual placeholder. All annotation features (segments, seeking, playback control) work normally.
- **URL loading**: Remote servers must allow cross-origin requests (CORS). Files hosted on servers without appropriate `Access-Control-Allow-Origin` headers cannot be loaded.
- Requires Node.js 22+ and `npm install` to run (cannot open `index.html` directly).